

Does AI Code Detection Reduce to *Skill* Detection?

A preliminary, two-sided answer

Autonomous experiment (Claude Code) for Matthew

May 20, 2026

Abstract

We ask whether AI-code detectors track author **species** (human vs. LLM) or author **skill/style**. Across 2655 Codeforces Python samples (human solutions stratified by problem difficulty; LLM solutions from 3 models under natural/novice/expert prompts) and three detectors spanning the trained (DroidDetect) and statistical (Fast-DetectGPT, Binoculars) families, the answer is two-sided. **(1)** On the competitive-programming difficulty axis the confound is *absent*: the skill-matched vs. mismatched AUROC gap is $\Delta_{\text{confound}} \approx 0.003$, and within-species score slopes run *against* the naive skill prediction (mean human slope -0.17 SD). This vindicates the concern that terse competitive-programming style is the stylistic *opposite* of the idiomatic “senior-engineer” register that RLHF code models imitate. **(2)** Yet on 135 samples of pre-LLM-era code from renowned engineers (Torvalds’ original `git`, antirez’s Redis, Hipp’s SQLite, CPython)—ironclad human and contamination-free—the trained detector labels **0.785** of it AI at its native threshold. The confound is therefore *real, but on the idiomatic-polish axis*, which competitive rating fails to capture.

1 Question and hypotheses

Post-RLHF code models are tuned toward idiomatic, well-structured “senior-engineer” style. If detectors learned that style as the AI signal, then *AI-vs-human* detection partly reduces to *expert-vs-novice* detection. We pre-registered:

- **H_{species}**: detectors classify species; within-species skill has no effect (slopes ≈ 0 ; AUROC constant across skill cells).
- **H_{skill}**: detectors classify skill; among humans, higher skill $\rightarrow$ more “AI”; among LLMs, novice prompting $\rightarrow$ less “AI”.
- **H_{mixed}**: a weighted combination.

2 Data and methods

Human. Accepted Python-3 Codeforces solutions (MatrixStudio), difficulty bands easy (≤ 1200) and hard (≥ 1900). *Deviation from the original plan*: no public dataset co-locates human source code with *author* Elo, so the human skill axis is problem difficulty—a noisy, and as it turns out stylistically *inverted*, proxy. **LLM.** 3 models (gemma-2-2b-it, Qwen2.5-3B-Instruct, and the frontier gpt-5.4-nano) under natural/novice/expert prompts on the same problems. **Detectors** (oriented higher = more AI): DroidDetect-Base-Binary (trained); Fast-DetectGPT and Binoculars (statistical; canonical vendored implementations). **Stats.** AUROC is primary (threshold-free); all CIs use

a cluster bootstrap over *problems*; the within-species score slope (z-scored detector output regressed on the skill axis) is the mechanism test; Δ_{confound} is reported per detector.

3 Results

3.1 The confound is absent on the competitive-skill axis

Table 1 gives within-species slopes (SD units). Under H_{skill} both columns should be positive; instead the human slope is *negative* for DroidDetect (-0.29 , CI $[-0.52, -0.06]$) and Fast-DetectGPT (-0.30), and null for Binoculars (0.07). Higher competitive-programming difficulty skill is scored, if anything, *less* AI-like.

Detector	Family	Human slope (hard–easy)	LLM slope (novice→expert)
DroidDetect	trained	-0.29	-0.09
Fast-DetectGPT	statistical	-0.30	-0.54
Binoculars	statistical	0.07	-0.19

Table 1: Within-species skill slopes (z-scored detector score). Positive = skill-tracking.

The skill-matched vs. mismatched AUROC gap is essentially zero (Table 2): $\Delta_{\text{confound}} = 0.011$ (CI $[0.002, 0.021]$) for DroidDetect, and -0.004 and 0.002 for the statistical detectors—i.e. matching humans and LLMs on competitive skill does *not* collapse detector performance. On this axis the data favour H_{species} .

Detector	Family	AUROC matched	AUROC mismatched	Δ_{confound}
DroidDetect	trained	0.741	0.752	0.011
Fast-DetectGPT	statistical	0.704	0.700	-0.004
Binoculars	statistical	0.665	0.667	0.002

Table 2: AUROC by skill cell and Δ_{confound} (cluster-bootstrap over problems).

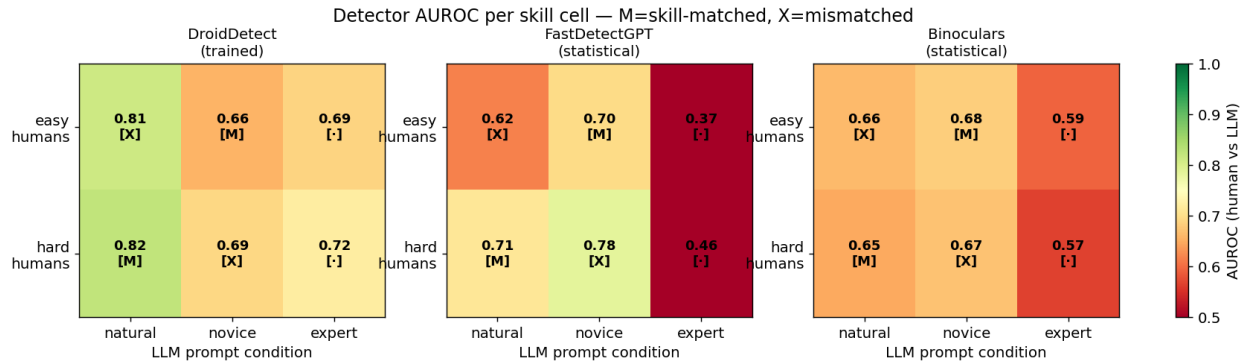


Figure 1: AUROC per (detector, difficulty band, prompt condition). M = skill-matched, X = mismatched.

3.2 But the confound is real on the idiomatic-polish axis

On contamination-free pre-LLM elite code, DroidDetect labels 0.785 of samples AI at its native threshold, and 0.489 even at a threshold calibrated to a 5% false-positive rate on ordinary Codeforces humans—roughly an order of magnitude above nominal. The effect is strongest on the most idiomatic code: CPython library code is flagged at 0.875 (5% FPR), versus 0.400 for Torvalds’ original `git` and 0.350 for Redis.

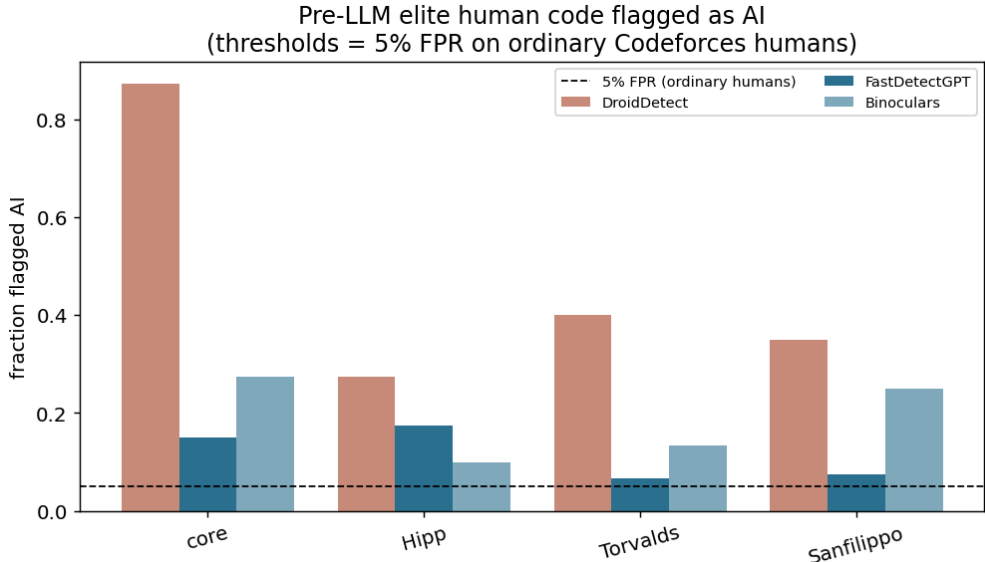


Figure 2: Pre-LLM elite human code flagged as AI (thresholds = 5% FPR on ordinary humans).

3.3 Mechanism and confirmatory checks

Surface style features explain $R^2 = 0.128$ of DroidDetect’s score and 0.276 of Fast-DetectGPT’s—style matters but does not fully account for the trained detector. On the harder Codeforces problems human code is in fact *more* idiomatic (type-hint rate $0.594 \rightarrow 0.755$), which makes the negative human slope notable: more idiomatic *competitive* code is still not read as more AI.

A confirmatory cohort using real *author* Elo (AtCoder; low band $n = 45$ mean Elo 323, high band $n = 9$ mean Elo 2225) points the same way: the high–low score gap is negative for all detectors (e.g. DroidDetect -0.37 SD), though the thin high band leaves wide CIs.

4 Limitations

- **Problem difficulty / competitive rating $\neq$ idiomatic-code skill.** This is now the central finding, not just a caveat: the proxy is stylistically inverted.
- **Accepted-only human code** is a collider (rating $\rightarrow$ accept $\leftarrow$ difficulty); LLM code is unfiltered—asymmetric.
- **Small LLM panel** (gemma-2-2b, Qwen2.5-3B, gpt-5.4-nano); preliminary n .
- **DroidDetect contamination** (Codeforces-like training data) not audited; the pre-LLM legendary probe is the contamination-free anchor.



Figure 3: Within-species slopes: detector score vs. skill, per species.

- **Thin AtCoder high-skill band** ($n = 9$).

5 Conclusion

“Does AI-code detection reduce to skill detection?” *Not* to competitive-programming skill—there the confound is absent and the relationship mildly inverts, because terse competitive style is the opposite of idiomatic style. But detection *is* materially confounded by **idiomatic code style**: a trained SOTA-style detector flags 0.785 of provably-human, pre-LLM, elite code as machine-generated. The deployment risk is real—the most polished human engineers are the most likely to be falsely accused—but the right axis to study it is production-code idiom, not competitive rating. The clean follow-up is an author-reputation cohort on idiomatic production code (e.g. GitHub by maintainer reputation) and a larger high-skill sample.